WEEK 2 | DAY 8

Document Ingestion & Chunking

RAG foundations: turning raw documents into structured, searchable knowledge

TOPICS

PDF and text parsing • cleaning and normalization • fixed-size, recursive and semantic chunking • metadata • contextual and parent-child chunking

Complete theory notes • Implementation intentionally excluded

Learning objectives

By the end of this lesson, you should be able to explain how raw documents become searchable knowledge in a Retrieval-Augmented Generation (RAG) system, compare major chunking strategies, and design metadata that preserves traceability.

- Explain why RAG is useful and distinguish indexing from query-time retrieval and generation.
- Describe text-based PDF parsing, scanned-PDF OCR, and plain-text parsing.
- Explain cleaning, normalization, and why document structure must be preserved.
- Compare fixed-size, recursive, and semantic chunking, including overlap.
- Explain document IDs, chunk IDs, source, page numbers, headings, and other metadata.
- Recognize contextual chunking, parent-child chunking, and common ingestion failure modes.

1. Why RAG is needed

Imagine a company has hundreds of internal policy documents, technical manuals, and employee handbooks. An employee asks: “What is the reimbursement limit for international travel?” A general-purpose LLM may not know the company's private or recently updated policy. Sending every document to the model for every question is expensive, slow, and often impossible because of context-window limits.

Retrieval-Augmented Generation (RAG) combines retrieval with generation. The system finds relevant information in an external knowledge source and supplies that information to an LLM as context when answering a question. A typical RAG answer is grounded in retrieved passages rather than relying only on the model's pretrained knowledge. RAG can reduce hallucinations and support source citations, but it does not guarantee correctness: the retriever may find the wrong passage, and the LLM may still misinterpret the evidence.

Why use RAG?

- **Private knowledge:** retrieve information from company or personal documents that was not part of model training.
- **Fresh information:** re-index updated documents without retraining the LLM.
- **Context efficiency:** send relevant passages instead of an entire collection.
- **Traceability:** preserve source information so answers can point back to a document and page.
- **Access control:** filter retrieval to content the current user is permitted to see.

2. The two major RAG pipelines

A RAG system usually has an indexing pipeline and a query-time retrieval-and-generation pipeline. They solve different problems and typically run at different times.

A. Indexing pipeline (document preparation)

1. **Collect and load:** identify files such as PDFs, TXT, DOCX, HTML, or spreadsheets.
2. **Parse:** extract text and, where needed, tables, headings, page information, or OCR output.
3. **Clean and normalize:** remove extraction noise while preserving useful structure.
4. **Chunk:** divide the content into smaller, manageable passages.
5. **Attach metadata:** retain document identity, source, page, section, and chunk position.

6. **Generate embeddings:** convert each chunk into a numerical vector using an embedding model.
7. **Store:** save text, vector, and metadata in a searchable index or vector database.

B. Query pipeline (answering a question)

1. **Receive** a user query.
2. **Embed** the query using a compatible embedding model.
3. **Retrieve** candidates by searching for vectors similar to the query vector, often with metadata filters.
4. **Prepare context** from retrieved chunks and metadata.
5. **Construct a prompt** containing the question and relevant evidence.
6. **Generate** an answer with the LLM, optionally including citations.

This lesson focuses on the indexing pipeline, particularly parsing, cleaning, chunking, and metadata. Embeddings, vector storage, retrieval strategies, and generation follow in later lessons.

3. Document ingestion

Document ingestion is the process of taking raw documents from different sources and converting them into a standardized representation that downstream components can process. It is more than opening a file: the pipeline must extract content, preserve meaningful structure, and retain enough information to trace every resulting chunk back to its source.

Typical stages

- **Loading:** locate files and choose a suitable loader or parser.
- **Extraction:** obtain text and relevant structural elements.
- **Cleaning:** remove unwanted artifacts and normalize formatting.
- **Structure preservation:** retain headings, paragraphs, lists, tables, and page boundaries when possible.
- **Chunk preparation:** produce a clean, structured representation ready for splitting.

4. Parsing document formats

A. PDF parsing

PDF means Portable Document Format. A PDF describes how content is laid out on pages; it is not necessarily a simple stream of paragraphs. It may contain selectable text, images, tables, headers, footers, multiple columns, diagrams, and scanned pages.

Two important PDF categories

PDF type	What it contains	Typical extraction approach
Text-based PDF	Text characters stored in the PDF and usually selectable.	Direct text extraction with a PDF parser.
Scanned PDF	Page images, often created from a physical scan; may contain no selectable text.	OCR to recognize text in the images.

Common Python tools

Tool	Typical use
PyMuPDF (fitz)	Extract text, images, and page-level information.
pypdf	Read and manipulate PDF files; extract text from supported PDFs.
pdfplumber	Extract text and tables with layout-oriented information.
Tesseract, EasyOCR, PaddleOCR	Recognize text in scanned pages or images.

OCR (Optical Character Recognition) identifies characters in an image and converts them into machine-readable text. OCR output is not guaranteed to be perfect: low resolution, skew, unusual fonts, handwriting, and complex layouts can introduce errors.

PDF layout is a major challenge. A two-column paper may be extracted in the wrong reading order. A table may become a sequence of disconnected words and numbers. A robust ingestion pipeline should preserve or reconstruct relationships between headings, paragraphs, rows, and columns wherever possible.

B. Plain-text files

TXT files are generally simpler because they contain plain text rather than a complex page layout. However, ingestion still needs to handle character encoding (such as UTF-8 or UTF-16), newline conventions, special characters, and excessive whitespace. Reading a UTF-16 file as UTF-8, for example, can produce decoding errors or garbled content.

C. Other formats

Format	Content to consider
DOCX	Paragraphs, heading hierarchy, tables, lists.
HTML	Main content, headings, links, navigation and boilerplate.
CSV / XLSX	Headers, rows, columns, sheets, and cell relationships.
PPTX	Slide text, titles, notes, and slide order.
Images	OCR text and, for visual applications, image content.

Parsing makes content machine-processable; it does not, by itself, make the content searchable. Chunking and indexing are subsequent steps.

5. Text cleaning and normalization

Extracted text often contains artifacts: repeated spaces, multiple blank lines, form-feed characters, broken symbols, duplicated page headers, or page numbers mixed into paragraphs. Cleaning makes the content more consistent before chunking.

Common operations

- **Whitespace normalization:** standardize repeated spaces, tabs, and blank lines.
- **Artifact removal:** remove unwanted control characters or extraction debris.
- **Header/footer handling:** detect repeated boilerplate that adds noise.

- **Encoding handling:** decode text correctly and preserve valid characters.
- **Structure preservation:** keep meaningful headings, numbered sections, bullets, and table relationships.

Do not over-clean. Removing a section heading may make the paragraph beneath it ambiguous. Deleting punctuation or table structure can change meaning. The objective is to remove noise while preserving information.

6. What is chunking?

Chunking divides a large document into smaller pieces called chunks. A 100-page document may contain many topics. Embedding and retrieving the entire document as one unit is often inefficient: the input may be too large, the relevant passage may be buried in unrelated content, and retrieval becomes less precise.

> **EXAMPLE SOURCE TEXT**
> **Company Travel Policy**
> Section 1: Domestic Travel
> Employees can claim accommodation expenses up to Rs. 3000 per day.
> Section 2: International Travel
> Employees can claim accommodation expenses up to Rs. 8000 per day.
> Section 3: Transportation
> Employees can claim taxi expenses with appropriate receipts.

A sensible split may produce one chunk for each section. A query about international travel can then retrieve the international-travel passage rather than the entire policy.

7. Fixed-size chunking

Fixed-size chunking divides text according to a target size. Size can be measured in characters, words, or tokens. Token-based sizing is useful because embedding models and LLMs have token limits. A token is a unit processed by a model: it may be a whole word, part of a word, punctuation, or another text fragment. One token is not necessarily one word.

Example: with a chunk size of five tokens and no overlap, the sequence “A B C D E F G H I J K L” becomes chunks “A B C D E”, “F G H I J”, and “K L”. The final chunk may be smaller.

Advantages

- Simple to implement and reason about.
- Predictable maximum size.
- Low computational overhead.
- Useful as a baseline for experiments.

Limitations

- May split a sentence or idea in the middle.
- May separate a heading from its paragraph.
- May separate a question from its answer or a table label from its value.
- Does not understand document meaning.

8. Chunk overlap

Overlap means that adjacent chunks share some text. If the chunk size is five tokens and overlap is two, one possible sequence is: Chunk 1 = A B C D E; Chunk 2 = D E F G H; Chunk 3 = G H I J K. The shared tokens provide continuity across boundaries.

Why use overlap?

A sentence or explanation may span a chunk boundary. Overlap makes it more likely that at least one retrieved chunk contains the full surrounding context. It is a technique that can be combined with fixed-size or other splitting approaches; it is not a separate core chunking algorithm.

Trade-off

More overlap	Less overlap
More boundary continuity and repeated context.	Less duplication and lower storage/embedding cost.
More tokens stored and potentially more duplicate retrieval.	Greater risk that related information is split apart.

A common experimental starting point might be a chunk size around 500 tokens with 50–100 tokens of overlap. These are not universal defaults; test them against the documents and questions in your application.

9. Recursive character text splitting

Recursive chunking tries to split text at meaningful structural separators before falling back to smaller separators. A typical hierarchy might be paragraph breaks, line breaks, sentence boundaries, word boundaries, and finally individual characters. The exact hierarchy depends on the splitter and its configuration.

How it works

1. Try splitting at a large boundary, such as a paragraph separator.
2. If a resulting piece exceeds the target size, split that piece using a smaller separator, such as a line break.
3. If it is still too large, try sentence or word boundaries.
4. As a last resort, split at a smaller unit such as characters.
5. Apply configured merge and overlap behaviour while keeping chunks within the target size as far as possible.

Advantages

- Usually preserves sentences and paragraphs better than arbitrary fixed-size splitting.
- Works well as a general-purpose baseline for mixed prose.
- Can be configured for different document structures.

Limitations

- Does not truly understand semantic meaning.
- Unrelated sentences can remain together if they share a paragraph.

- Related content can still be separated by document structure.
- Output depends on separator order and configuration.

> **Important distinction:** recursive chunking is structure-aware, not embedding-based semantic chunking. It tries sensible boundaries but does not measure meaning.

10. Semantic chunking

Semantic chunking attempts to divide text when its meaning or topic changes. It commonly begins by splitting a document into smaller units, such as sentences, then uses an embedding model to represent those units as vectors. Similarity between neighbouring units can help identify topic transitions.

> **ILLUSTRATIVE EXAMPLE**
> “The human body requires sufficient sleep.”
> “Sleep plays an important role in memory consolidation.”
> “Python is a popular programming language.”
> “Python supports object-oriented programming.”

A semantic strategy may group the first two sentences into a sleep-related chunk and the next two into a Python-related chunk, even though all four sentences appear consecutively in the same source.

Typical semantic chunking workflow

1. Split the text into small units, often sentences or short groups of sentences.
2. Generate an embedding for each unit or a rolling window of units.
3. Compare neighbouring embeddings using a similarity measure such as cosine similarity.
4. Identify unusually large changes in similarity as possible topic boundaries.
5. Group related units into chunks while enforcing practical minimum and maximum size constraints.

Cosine similarity

Cosine similarity compares the angle between two vectors:

```
cosine_similarity(A, B) = (A · B) / (||A|| × ||B||)
```

Its mathematical range is −1 to 1. For many text embedding models, semantically related passages tend to have higher scores, but score distributions and interpretation depend on the model.

A sharp drop in similarity between adjacent sentences can indicate a topic change. Real systems may use rolling windows, buffer sizes, percentile thresholds, or other boundary-detection methods. A score alone does not prove that a topic changed.

Advantages and limitations

Advantages	Limitations
Can group topically related passages.	Requires an embedding model during ingestion.
Can adapt boundaries to topic changes.	More computationally expensive than simple splitting.
May improve coherence for complex documents.	Can produce inconsistent chunk sizes and needs threshold tuning.
Useful when fixed structure does not match meaning.	Semantic similarity does not guarantee logical completeness.

11. Comparing the three main strategies

Property	Fixed-size	Recursive	Semantic
Main basis	Target size	Hierarchical separators	Meaning/topic change
Uses embeddings	No	No	Usually yes
Respects boundaries	Not guaranteed	Attempts to	Depends on method
Semantic understanding	No	No	Attempts to capture it
Complexity	Low	Medium	Higher
Typical role	Baseline	General-purpose starting point	Specialized strategy to evaluate

> **Practical recommendation:** begin with recursive chunking for a general-purpose RAG system. Consider semantic chunking when topic transitions or document structure make simpler splitting insufficient. Evaluate retrieval quality instead of assuming the most sophisticated method is automatically best.

12. Metadata preservation

Metadata is information about a piece of data. For a chunk, it describes where the text came from, where it appeared, and how it relates to the original document. Store the chunk text together with metadata, or in linked records that preserve the relationship.

EXAMPLE CHUNK METADATA

```
document_id: DOC_001
source: company_travel_policy.pdf
page_number: 5
chunk_id: DOC_001_CHUNK_03
section: International Travel
```

Important fields

Field	Purpose
document_id	Identifies the original document; shared by all its chunks.
chunk_id	Uniquely identifies one chunk.
source	Original filename, URI, or source location.
page_number / page_start / page_end	Locates the content in the source document.
section / heading	Preserves the topic or structural context.
chunk_index	Records the chunk's order within the document.
file_type	Identifies PDF, TXT, DOCX, and so on.
version / updated_at	Helps manage revisions and stale content.
department / category	Supports filtering and organization.
access_level	Can support authorization-aware retrieval when enforced by the application.

Why metadata matters

- **Source attribution:** cite the original file and page.
- **Filtering:** restrict search by category, department, date, or other attributes.
- **Reconstruction:** group chunks by document ID and order them by chunk index.
- **Access control:** retrieve only records the current user is allowed to access.

> **Security note:** storing an access-level field is not sufficient by itself. The application must enforce authorization during retrieval and before exposing content.

13. Preserve document and page boundaries

Do not accidentally merge the end of one document with the beginning of another. For example, “The company provides medical insurance” from Document A and “Employees are eligible after six months” from Document B should not become one apparent statement. Process documents independently unless cross-document combination is intentional, and retain document IDs and source references throughout the pipeline.

For PDFs, page boundaries are especially useful. A chunk may come from one page or span multiple pages. If it spans pages, preserve a page range or page references rather than assigning an inaccurate single page number.

14. Choosing chunk size

There is no universally optimal chunk size. The right value depends on document type, the embedding model's input limit, retrieval behaviour, the LLM's context budget, and the kind of questions users ask.

Illustrative size	Potential benefits	Potential risks
100–200 tokens	Fine-grained retrieval; less unrelated content.	May fragment explanations and lose context.
300–700 tokens	Often a useful balance for experimentation.	May still split related ideas or combine topics.
800–1,500 tokens	Preserves larger explanations and surrounding context.	May contain unrelated material and reduce precision.

If chunks are too small, the answer may be split across multiple records. If chunks are too large, retrieval may return a lot of irrelevant text. Treat chunk size and overlap as parameters to evaluate on representative questions.

15. Contextual chunking

A chunk may be technically complete but ambiguous when separated from its document. For example, “The limit is Rs. 8000 per day” does not say what limit it describes. Contextual chunking enriches the chunk with relevant surrounding information such as the document title, section heading, parent heading, table title, or introductory context.

EXAMPLE ENRICHED CHUNK

```
Document: Company Travel Policy
Section: International Travel
Topic: Accommodation reimbursement
The limit is Rs. 8000 per day.
```

Contextual chunking adds context to a chunk. Semantic chunking decides where to split based on meaning. A system can use both.

16. Parent-child chunking

Parent-child chunking stores or maintains larger parent passages alongside smaller child chunks. Retrieval searches the smaller child chunks for precision. Once a child matches, the system can fetch its parent section to give the LLM more surrounding context.

Example: a large “International Travel” section is the parent. Smaller children cover eligibility, accommodation limits, transport limits, and approval requirements. A query about accommodation may match the accommodation child, while the parent section supplies additional context.

This combines the precision of smaller retrieval units with the context of larger passages. It is useful when a specific fact depends on nearby explanations or conditions.

17. Common ingestion problems

Problem	Possible consequence	Mitigation
Scanned PDF	No selectable text.	OCR and quality checks.
Complex tables	Lost row/column relationships.	Layout-aware table extraction.

Problem	Possible consequence	Mitigation
Multi-column layout	Incorrect reading order.	Layout-aware parsing.
Repeated headers	Noisy or repetitive chunks.	Detect and remove boilerplate carefully.
Duplicate files	Duplicate chunks and retrieval results.	Hashes, IDs, and deduplication.
Poor OCR	Incorrect facts and numbers.	Quality checks and source review.
Missing metadata	No reliable source traceability.	Preserve metadata during parsing and chunking.
Broken boundaries	Incomplete or misleading passages.	Structure-aware splitting and overlap.
Very large files	High memory use or long processing time.	Batching and streaming where appropriate.
Updated documents	Stale indexed information.	Versioning and controlled re-ingestion.

A production ingestion pipeline should be repeatable and idempotent where possible. Re-ingesting the same unchanged document should not create uncontrolled duplicate chunks. Document hashes, stable IDs, versions, and cleanup policies can help.

18. End-to-end worked example

Suppose a university RAG assistant ingests `student_handbook.pdf`, a three-page PDF.

Page	Extracted content
1 — Attendance	Students must maintain a minimum attendance of 75%.
2 — Examinations	Students must complete all required assignments before appearing for the examination.
3 — Fees	Tuition fees must be paid before the specified deadline.

After parsing and cleaning, recursive chunking may produce three chunks, one per section. Each chunk receives a document ID, chunk ID, page number, source filename, section heading, and chunk index. Later, an embedding model converts each chunk into a vector, and the vector plus text and metadata are stored in a searchable index.

When a student asks “What is the minimum attendance requirement?”, retrieval can find the attendance chunk. The LLM receives the question and retrieved passage and can answer: “Students must maintain a minimum attendance of 75%,” with a source reference to student_handbook.pdf, page 1.

19. Placement interview questions

1. What is document ingestion in RAG?

Loading documents, extracting and cleaning their content, preserving useful structure and metadata, and preparing the content for chunking and indexing.

2. Why do we split documents into chunks?

To make information manageable for embedding and retrieval, improve retrieval granularity, and avoid sending entire documents to the LLM.

3. What is the difference between fixed-size and recursive chunking?

Fixed-size chunking splits according to a target size. Recursive chunking tries larger structural separators first and falls back to smaller separators when necessary.

4. What is semantic chunking?

It uses semantic relationships, often measured through embeddings and similarity, to identify topic changes and group related text.

5. Why do we use chunk overlap?

To preserve context across chunk boundaries and reduce the chance that splitting separates related information.

6. Why is metadata important in RAG?

It supports source attribution, filtering, traceability, document reconstruction, and authorization-aware retrieval.

7. Why is OCR required for scanned PDFs?

Scanned PDFs may contain page images rather than selectable text. OCR converts text in those images into machine-readable text.

8. What happens if chunk size is too large or too small?

Small chunks may lose context; large chunks may include unrelated information and reduce retrieval precision.

9. What is parent-child chunking?

It retrieves using smaller child chunks and can return their larger parent sections to provide additional context.

10. Does semantic chunking always produce better results?

No. It adds computational cost and complexity. Its effectiveness depends on the documents, embedding model, and retrieval evaluation.

20. Day 8 recap

Concept	Remember
RAG	Retrieve external knowledge and provide it to an LLM as context.
Ingestion	Convert raw documents into processable content.
PDF parsing	Extract text while preserving layout and structure.
OCR	Recognize text in scanned images.
Cleaning	Remove noise without destroying meaning.
Chunking	Divide documents into smaller retrieval units.
Fixed-size	Split according to a target size.
Recursive	Use hierarchical separators, falling back to smaller ones.
Semantic	Use semantic similarity to help identify topic changes.
Overlap	Share context between neighbouring chunks.
Metadata	Preserve identity, origin, location, and relationships.

Concept	Remember
Contextual chunking	Enrich chunks with surrounding document context.
Parent-child chunking	Retrieve small children and optionally return larger parent context.

What comes next in Week 2?

1. **Embeddings:** how text becomes vectors and what embedding models learn to represent.
2. **Vector databases:** storing, indexing, and searching embeddings.
3. **Retrieval:** similarity search, top-k, metadata filters, hybrid search, and reranking.
4. **RAG implementation:** connect retrieval to an LLM and generate grounded answers.
5. **LangGraph:** orchestrate stateful workflows using nodes, edges, and conditional routing.

Final learning advice

Do not rush into a framework before understanding what happens to a document before it reaches an embedding model. Parsing quality, chunk boundaries, and metadata directly affect retrieval quality. In implementation, compare strategies on the same documents and evaluate them using realistic questions.

End of Day 8 theory notes • No implementation code included